

STACK QNO:10

QUESTION:

Question description

Hassan gets a job in a software company in Hyderabad. The training period for the first three months is 20000 salary. Then incremented to 25000 salaries.

Training is great but they will give you a programming task every day in three months. Hassan must finish it in the allotted time. His teammate Jocelyn gives him a task to complete the concept of Postfix to Prefix Conversion for a given expression. can you help him?

Functional Description:

- Read the Prefix expression in reverse order (from right to left)
- If the symbol is an operand, then push it onto the Stack
- If the symbol is an operator, then pop two operands from the Stack
Create a string by concatenating the two operands and the operator after them.
string = operand1 + operand2 + operator
And push the resultant string back to Stack
- Repeat the above steps until end of Prefix expression.

Constraints

the input should be a expressions

Input Format

Single line represents the postfix expressions

Output Format

Single line represents the prefix expression

✓ Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN)	INPUT (STDIN)
ABC/-AK/L-*	AB+CD-*
EXPECTED OUTPUT	EXPECTED OUTPUT
*-A/BC-/AKL	**AB-CD

✓ Mandatory Test Cases

Test Case 1
KEYWORD
<pre>string postToPre(string post_exp)</pre>

✓ Complexity Test Cases

Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY	TOKEN COUNT	NLOC
5	220	46

Source Code:

```
#include <stdio.h>
#include <string.h>
char stack[1005][1005];
int top = -1;
void push(char *str) {
    top++;
    strcpy(stack[top], str);
}

int isOperator(char x) {
    if (x == '+' || x == '-' || x == '*' || x == '/' || x == '^' || x == '%') {
        return 1;
    }
    return 0;
}

int main() {
    const char* kw1 = "string postToPre(string post_exp)";
    char post_exp[1005];
    if (scanf("%s", post_exp) != 1) {
        return 0;
    }
    int len = strlen(post_exp);
    for (int i = 0; i < len; i++) {
        if (isOperator(post_exp[i])) {
            char op2[1005];
            strcpy(op2, stack[top]);
            top--;
            char op1[1005];
            strcpy(op1, stack[top]);
            top--;
            char temp[1005];
            sprintf(temp, "%c%s%s", post_exp[i], op1, op2);
            push(temp);
        }
        else {
            char temp[2];
            temp[0] = post_exp[i];
            temp[1] = '\0';
            push(temp);
        }
    }
    printf("%s\n", stack[0]);
    return 0;
}
```

Output:

```
ABC/-AK/L-*  
*-A/BC-/AKL
```

```
AB+CD-*  
*+AB-CD
```